DATE	28-02-2026
TEAM ID	LTVIP2026TMIDS***
PROJECT NAME	HEMATOVISION: ADVANCED BLOOD CELL CLASSIFICATION USING TRANSFER LEARNING
MAXIMUM MARKS	4 MARKS

6.4 MODEL BUILDING :

Model Building:

Mobilenet V2 Transfer-Learning Model:

The MobileNetV2-based neural network is created using a pre-trained MobileNetV2 architecture with frozen weights. The model is built sequentially, incorporating the MobileNetV2 base, a flattening layer, dropout for regularization, and a dense layer with SoftMax activation for classification into four categories of blood cells. The model is compiled using the Adam optimizer and categorical cross-entropy loss. During training, which spans 5 epochs, a generator is employed for the training data, and validation is conducted with callbacks such as Model Checkpoint and Early Stopping. The best-performing model is saved as "blood_cell.h5" for future use. The model summary provides an overview of the architecture, showcasing the layers and parameters involved.

The HematoVision model is developed using a transfer learning approach based on the MobileNetV2 architecture. A pre-trained MobileNetV2 model, trained on large-scale image datasets, is used as the base model to leverage previously learned features. The weights of the base model are frozen to retain its learned feature extraction capabilities and reduce training time.

The model is constructed sequentially by adding the MobileNetV2 base, followed by a Flatten layer to convert feature maps into a one-dimensional vector. A Dropout layer is included to prevent overfitting and improve generalization. Finally, a Dense output layer with SoftMax activation is added to classify images into four different blood cell categories.

The model is compiled using the Adam optimizer, which ensures efficient weight updates, and categorical cross-entropy as the loss function for multi-class classification. During training, a data generator is used to feed augmented training images to the model, improving robustness and accuracy.

Training is performed for 5 epochs with validation data to monitor performance. Callbacks such as Model Checkpoint and Early Stopping are implemented to save the best-performing model and prevent overfitting. The final optimized model is saved as "blood_cell.h5" for future predictions and deployment. The model summary provides a detailed overview of the architecture, including layers, output shapes, and total parameters involved in the network.

```
import tensorflow as tf
import numpy as np
import os
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras import layers, models
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from sklearn.utils.class_weight import compute_class_weight

# ✅ Your Correct Dataset Path
data_dir = r'D:\PythonProject\static\dataset2-master\dataset2-master\images\TRAIN'

# Confirm path exists
print("Dataset Found ✅")
print("Folders:", os.listdir(data_dir))

# Data Augmentation
datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    zoom_range=0.2,
    horizontal_flip=True,
    validation_split=0.2
)

train_data = datagen.flow_from_directory(
    data_dir,
    target_size=(224,224),
    batch_size=32,
    class_mode='categorical',
    subset='training'
)

val_data = datagen.flow_from_directory(
    data_dir,
    target_size=(224,224),
    batch_size=32,
    class_mode='categorical',
    subset='validation'
```

```
)

print("Class Indices:", train_data.class_indices)

# Handle Class Imbalance
class_weights = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(train_data.classes),
    y=train_data.classes
)

class_weights = dict(enumerate(class_weights))

# EfficientNet Model
base_model = EfficientNetB0(
    input_shape=(224,224,3),
    include_top=False,
    weights='imagenet'
)

base_model.trainable = True

# Freeze first 100 layers
for layer in base_model.layers[:100]:
    layer.trainable = False

model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.BatchNormalization(),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(4, activation='softmax')
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
```

```
)

callbacks = [
    EarlyStopping(patience=5, restore_best_weights=True),
    ReduceLROnPlateau(patience=3)
]

history = model.fit(
    train_data,
    validation_data=val_data,
    epochs=25,
    class_weight=class_weights,
    callbacks=callbacks
)

model.save("blood_cell_model.h5")

print(" Training Complete & Model Saved!")
```